

Python Static Security Analyzer

Academic Project Documentation and Source Code

1. Project Overview

This project implements a lightweight static security analyzer for Python source code. The program examines a Python file without executing it, parses the source code into an Abstract Syntax Tree (AST), and searches for selected security and code-quality issues.

The analyzer currently identifies four main categories of findings:

- Hard-coded passwords or password variables — reported as **MEDIUM** severity.
- Use of `eval()` and `exec()` — reported as **HIGH** severity.
- Use of `os.system()` and `subprocess.call()` — reported as **HIGH** severity.
- Assigned variables that are not subsequently used — reported as **LOW** severity.

2. Methodology

The scanner follows a static-analysis workflow. First, it receives the target Python file through the command line. It then reads the source code and passes it to Python's built-in `ast` module. The resulting Abstract Syntax Tree is traversed using `ast.walk()`. During traversal, the program inspects assignments, function calls, and variable references. Detected findings are stored with their line number, severity level, vulnerability type, and descriptive message.

3. Main Components

AST Parsing: Converts Python source code into a structured tree so the analyzer can inspect program constructs rather than relying only on text matching.

Security Rules: Detects selected dangerous calls and possible hard-coded credentials.

Variable Analysis: Tracks assigned and used variable names to identify simple unused-variable cases.

Severity Classification: Findings are categorized as HIGH, MEDIUM, or LOW.

Terminal Reporting: Uses ANSI escape codes to display severity levels in different terminal colors.

File Reporting: Creates a clean *report.txt* file containing the detected findings and generation time.

4. How to Run

```
python scanner.py <file.py>
```

The analyzer is intended to be run from a terminal. The input argument specifies the Python source file to scan. If the file contains invalid Python syntax, the program reports the syntax error and stops instead of continuing with an invalid AST.

5. Example Detection Logic

For example, if the scanned program contains **eval(user_input)**, the analyzer recognizes the call as a potentially dangerous operation and produces a HIGH-severity warning. Similarly, a variable named **password** assigned directly in the source is reported as a potential hard-coded secret.

6. Limitations

This implementation is a lightweight educational static analyzer rather than a complete security auditing framework. Its rules are pattern-based and therefore may produce false positives or miss vulnerabilities that require deeper data-flow or control-flow analysis. For example, it does not currently determine whether a value assigned to a variable actually contains a secret, nor does it perform taint analysis to track untrusted input into dangerous sinks.

7. Academic Significance

The project demonstrates practical application of static program analysis, abstract syntax trees, source-code inspection, vulnerability classification, and automated reporting. It also provides a foundation that can be extended with additional security rules, data-flow analysis, configuration files, severity scoring, and integration into software-development workflows.

8. Complete Source Code

```
import ast
import sys
import time
import builtins

# ■ Colors
RED = "\033[91m"
YELLOW = "\033[93m"
GREEN = "\033[92m"
RESET = "\033[0m"

# check input file
if len(sys.argv) < 2:
    print("Usage: python scanner.py <file.py>")
    exit()

filename = sys.argv[1]

print("Scanning:", filename)

# read file safely
with open(filename, "r", encoding="utf-8", errors="ignore") as f:
    code = f.read()

# parse AST safely
try:
    tree = ast.parse(code)
except SyntaxError as e:
    print("Syntax Error in file:", e)
    exit()

# reports
file_report = []
file_report.append("==== SECURITY REPORT =====\n")
file_report.append(f"Generated at: {time.ctime()}\n\n")

assigned = {}
used = set()
warnings = []

# scan AST
for node in ast.walk(tree):

    # assignments
    if isinstance(node, ast.Assign):
        for target in node.targets:
            if isinstance(target, ast.Name):
                assigned[target.id] = node.lineno

                if target.id.lower() == "password":
                    warnings.append((
                        node.lineno,
                        "MEDIUM",
                        "HARD_CODED_SECRET",
                        f"[MEDIUM] Hardcoded Password at line {node.lineno}\n"
                    ))

    # function calls
    if isinstance(node, ast.Call):

        # direct calls
        if isinstance(node.func, ast.Name):

            if node.func.id == "eval":
                warnings.append((
                    node.lineno,
                    "HIGH",
                    "DANGEROUS_CALL",
                    f"[HIGH] Dangerous function eval() at line {node.lineno}\n"
                ))

            if node.func.id == "exec":
                warnings.append((
                    node.lineno,
                    "HIGH",
                    "DANGEROUS_CALL",
                    f"[HIGH] Dangerous function exec() at line {node.lineno}\n"
                ))

        # attribute calls
        if isinstance(node.func, ast.Attribute):
            if isinstance(node.func.value, ast.Name):

                if node.func.value.id == "os" and node.func.attr == "system":
                    warnings.append((
                        node.lineno,
                        "HIGH",
```

```

        "DANGEROUS_CALL",
        f"[HIGH] Dangerous function os.system() at line {node.lineno}\n"
    ))

    if node.func.value.id == "subprocess" and node.func.attr == "call":
        warnings.append((
            node.lineno,
            "HIGH",
            "DANGEROUS_CALL",
            f"[HIGH] Dangerous function subprocess.call() at line {node.lineno}\n"
        ))

# track used variables
if isinstance(node, ast.Name):
    if isinstance(node.ctx, ast.Load):
        if node.id not in dir(builtins):
            used.add(node.id)

# unused variables
unused = set(assigned.keys()) - used

for var in unused:
    if var.lower() != "password":
        line = assigned[var]
        warnings.append((
            line,
            "LOW",
            "UNUSED_VAR",
            f"[LOW] Unused variable: {var} (defined at line {line})\n"
        ))

# sort by line number
warnings.sort(key=lambda x: x[0])

# output
for _, severity, wtype, msg in warnings:

    # terminal colors
    if severity == "HIGH":
        print(RED + msg + RESET, end="")
    elif severity == "MEDIUM":
        print(YELLOW + msg + RESET, end="")
    else:
        print(GREEN + msg + RESET, end="")

    # save to file (clean format)
    file_report.append(f"{severity} - {wtype} - {msg}")

print("\n===== END REPORT =====")

file_report.append("\n===== END REPORT =====\n")

# save report
with open("report.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(file_report))

```